14 haftalık yol haritasında 9. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 09

Fonksiyonlar II: Kapsam, Ömür, Çağrı Yığını ve Özyinelemeye Giriş

Blok, fonksiyon ve dosya kapsamı; static; çok dosyalı derleme ve makrolar; çağrı yığını, süreç bellek düzeni ve yığın taşması; özyineleme, Fibonacci ve Hanoi Kuleleri

MODÜL IV · MODÜLER PROGRAMLAMA · H8–H9

> HAFTANIN BÜYÜK SORUSU
>
> *Bir fonksiyon kendini çağırırsa ne olur ve bilgisayar bu çağrıların hangisinden nereye döneceğini nasıl hatırlar?*

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları 3
1.1 Öğrenme çıktıları 3
2. Geçen Haftadan: Çağrıdan Çağrı Yığınına 3
3. Kavramsal Anlatım 4
3.1 Kapsam: bir ad nerede görünür? 4
3.2 Global değişkenler ve yan etki riski 5
3.3 Ömür: otomatik ve statik 5
3.4 Çok dosyalı programlar: .h, .c, extern, static ve başlık koruyucusu 6
3.5 Önişlemci makroları ve tuzakları 7
3.6 Kaputun altında: çağrı yığını ve süreç bellek düzeni 8
3.7 Özyineleme: problemi kendi küçük örneğine indirgemek 9
3.8 Yinelemeli ↔ özyinelemeli dönüşüm 10
4. Çözümlü Örnekler: Örnek Merdiveni 11
4.1 ★ basamaklar: kapsam, static ve ilk özyineleme (Örnek 1–3) 11
4.2 Örnek 4 ★★ — Çok dosyalı proje: matematik_util 11
4.3 Örnek 5 ★★ — KARE(x) makro tuzağı 12
4.4 Amiral örnek — Örnek 6 ★★: özyinelemeli faktöriyel ve yığın çizimi 12
4.5 Örnek 7 ★★ — 1..N toplamı ve üs alma 14
4.6 Örnek 8 ★★ — Basamak toplamı ve “dönüşte iş yapma” 15
4.7 Amiral örnek — Örnek 9 ★★: onluk sayıyı ikili yazdırma 16
4.8 Örnek 10 ★★ — Özyinelemeli EBOB (Öklid) 17
4.9 Örnek 11 ★★ — Sonsuz özyineleme ve yığın taşması 17
4.10 Örnek 12 ★★★ — Özyinelemeli Fibonacci ve çağrı sayısı 18
4.11 Örnek 13 ★★★ — Hanoi Kuleleri 18
5. Hata Avcısı 19
6. Mühendis Gözüyle: Kapsamı Daralt, Yığını Hesapla 23
7. Mini-OBS v0.9: Çok Dosyalı Modüler Yapı 23
8. Alıştırmalar ve Atölye 25
9. Kendini Değerlendirme 28
10. AP-2 Köprüsü ve Merak Kutusu 28
11. Özet, Kavram Haritası ve Terim Sözlüğü 29
12. Kaynaklar 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir fonksiyon kendini çağırırsa ne olur ve bilgisayar bu çağrıların hangisinden nereye döneceğini nasıl hatırlar?

H8'de `main` bir fonksiyonu çağırıyor, fonksiyon işini yapıp **geri dönüyordu**. Dönüş adresi nerede saklanıyor? Bir fonksiyon **kendini** çağırırsa her çağrının kendi `n`'si mi olur? Bu hafta yanıtı bellekte arıyoruz: adın **nerede görünür** olduğu (kapsam), değişkenin **ne kadar yaşadığı** (ömür) ve her çağrının yığında açtığı **çerçeve**. Bu modelle **özyinelemeyi** (recursion), yani bir problemi kendi küçük örneğine indirgeyerek çözmeyi öğreneceğiz.

> GERÇEK DÜNYADAN · **1883: Hanoi Kuleleri ve 64 altın disk**
>
> **Édouard Lucas** (1842–1891), Hanoi Kuleleri bulmacasını 1883'te, "Lucas d'Amiens"in harf oyunu olan **N. Claus de Siam** takma adıyla yayımladı (Hinz vd., 2018; Merrotsy, 2015). Kural: her hamlede yalnızca en üstteki disk taşınır, hiçbir disk kendisinden küçüğünün üstüne konamaz. Bulmacayla birlikte bir **efsane anlatılır**: tapınaktaki rahipler 64 altın diskli kuleyi taşıyınca dünyanın sonu gelecektir (Merrotsy, 2015).
>
> Efsane uydurmadır ama sorusu gerçektir: **kaç hamle gerekir?** Beş satırlık özyinelemeli fonksiyonumuzun yanıtı: n disk için en az $2^n - 1$; 64 disk için `18 446 744 073 709 551 615` hamle. Saniyede bir hamleyle `H09_K10_hanoi_64.c`'nin hesabı yaklaşık **5,85 × 10^{11} yıl**: evrenin yaşının (≈ 13,8 milyar yıl; Planck Collaboration, 2020) yaklaşık **42,4 katı**.

1.1 Öğrenme çıktıları

ÖÇ 9.1 Blok, fonksiyon ve dosya kapsamını; otomatik ve statik ömrü ayırt eder. *(Bölüm 3.1–3.3, Örnek 1–2)*

ÖÇ 9.2 Çağrı yığınının çalışmasını yığın çerçeveleri üzerinden çizerek açıklar. *(Bölüm 3.6, 4.4 (Örnek 6))*

ÖÇ 9.3 Özyinelemeli bir fonksiyonun temel durumunu ve özyinelemeli adımını tanımlar; özyineleme ağacı ile yürütmeyi izler. *(Bölüm 3.7, 4.4–4.11)*

ÖÇ 9.4 Basit problemleri hem yinelemeli hem özyinelemeli çözer; iki yaklaşımı bellek ve zaman açısından karşılaştırır. *(Bölüm 3.8, Örnek 7–12)*

ÖÇ 9.5 Başlık (.h) ve kaynak (.c) dosyalarından oluşan çok dosyalı bir C projesi oluşturur ve derler. *(Bölüm 3.4, 4.2, Bölüm 7)*

2. Geçen Haftadan: Çağrıdan Çağrı Yığınına

H8'de prototipi, değerle aktarımı ve `assert` ile birim testini öğrendik; çağrıda "argümanların kopyalandığını ve yığına bir çerçeve eklendiğini" giriş düzeyinde söyledik. Bu hafta ayrıntısına iniyoruz. Önce üç geri çağırma sorusu (`kodlar/quiz/`):

> TAHMİN ET · **Geri çağırma: üç soru**
>
> - **S1 (prototip).** `H09_Q01_prototip.c`'de `kup`'un tanımı `main`'den **sonra**, prototipi önce. Ne yazdırır? Prototip silinirse?
> - **S2 (değerle aktarım).** `H09_Q02_takas.c`'de `takas(x, y)` çağrısından sonra `x` ve `y` kaç olur?
> - **S3 (assert).** `H09_Q03_assert.c`'nin üçüncü testi `assert(mutlak(7) == -7)`. Ne yazdırır, çıkış durumu kaç?

Yanıtlar (gerçek). S1: "kup(3) = 27"; prototip silinirse GCC 13 "implicit declaration of function 'kup' [-Wimplicit-function-declaration]" uyarısı verir. S2: "x = 1, y = 2": `takas` kendi **çerçevesindeki kopyaları**

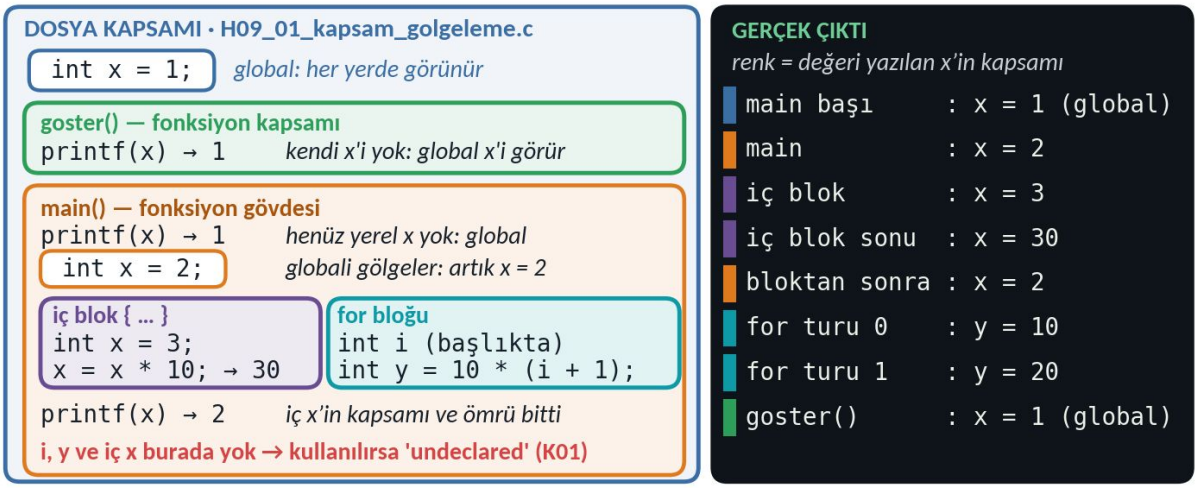
değiştirir; çerçeve dönüşte yok olur (çözüm H12). S3: “q03: H09_Q03_assert.c:20: main: Assertion `mutlak(7) == -7' failed.”; kabuk “Aborted” bildirir, çıkış durumu **134** (128 + 6, SIGABRT).

3. Kavramsal Anlatım

3.1 Kapsam: bir ad nerede görünür?

Bir adın **kapsamı** (scope), kaynak kodda **kullanılabildiği bölgedir** (ISO/IEC, 2018, 6.2.1; King, 2008, Bölüm 10). **Blok kapsamı:** `{ }` içinde bildirilen ad, bildirildiği yerden bloğun sonuna kadar görünür; parametreler ve `for (int i = …)` sayacı da blok kapsamlıdır. **Dosya kapsamı:** bütün fonksiyonların dışında bildirilen ad (global) dosyanın sonuna kadar her fonksiyonda görünür. (C’deki asıl **fonksiyon kapsamı** yalnızca `goto` etiketleri içindir; “fonksiyonun kapsamı” dediğimiz şey gövdenin bloğudur.) İç blokta **aynı ad** yeniden bildirilirse içteki, dıştakini kendi bloğu boyunca **gölgeler** (shadowing). Örnek 1 aynı `x` adını üç değişkene verir (Şekil 9.1):

Şekil 9.1
Örnek 1’in kapsam bölgeleri iç içe kutular olarak; sağda gerçek çıktı, kapsam rengiyle



H09_01_kapsam_golgeleme.c · satır 9–36

```c
#include <stdio.h>

int x = 1;                          /* dosya kapsamı (global) */

void goster(void)
{
    printf("goster()      : x = %d (global)\n", x);
}

int main(void)
{
    printf("main başı     : x = %d (global)\n", x);
    int x = 2;                      /* main'in x'i: globali gölgeler */
    printf("main          : x = %d\n", x);
    {
        int x = 3;                  /* iç blok: main'in x'ini gölgeler */
        printf("iç blok       : x = %d\n", x);
        x = x * 10;                 /* yalnızca iç x değişir */
        printf("iç blok sonu  : x = %d\n", x);
    }                               /* iç x'in kapsamı ve ömrü biter */
```

```
29      printf("bloktan sonra : x = %d\n", x);
30      for (int i = 0; i < 2; i++) {
31          int y = 10 * (i + 1);       /* her turda yeniden oluşur */
32          printf("for turu %d    : y = %d\n", i, y);
33      }
34      goster();                       /* goster() main'in x'ini görmez */
35      return 0;
36  }
```

Ad **en içteki** bloktan dışarıya doğru aranır. 20. satırda yerel `x` henüz yoktur: global (1) yazılır; 21. satırdan sonra `main`'in `x`'i (2) globali gölgeler. İç bloktaki `x = x * 10` yalnızca iç `x`'i değiştirir (30). `goster()` `main`'in içini göremez ve global `x`'i (1) yazar. Gölgeleme yasal ama tehlikelidir; `-Wshadow` bildirir: "declaration of 'x' shadows a global declaration [-Wshadow]" ve "declaration of 'x' shadows a previous local [-Wshadow]". Kapsamı bitmiş bir adı kullanmak ise **derleme hatasıdır**: `H09_K01_kapsam_disi.c` (KASITLI HATA) `for`'dan sonra `i`'yi yazdırmaya çalışınca GCC "'i' undeclared (first use in this function)" der. Kural: değişkeni **en dar kapsamda** bildirin.

3.2 Global değişkenler ve yan etki riski

Global değişkeni dosyadaki **her fonksiyon** okuyup değiştirebilir: fonksiyonun davranışı artık başkalarının o kutuya ne yazdığına da bağlıdır, H8'in kara kutu yaklaşımı delinir (HA3'te iki fonksiyonun paylaştığı global sayaç üç turluk döngüyü tek turda bitirir). Program boyu yaşaması gereken durum için daha güvenli araç `static`'tir.

3.3 Ömür: otomatik ve statik

Ömür (lifetime; standartta saklama süresi) değişkenin bellekte **ne zaman var olduğunu** anlatır; kapsam ise **nerede görünür** olduğunu (ISO/IEC, 2018, 6.2.4). Sıradan yerel değişkenin ömrü **otomatiktir**: bloğa girişte oluşur, çıkışta yok olur, ilk değer verilmezse değeri belirsizdir. `static` ile bildirilen yerel değişkenin ömrü **statiktir**: program başlarken **bir kez** oluşur (verilmezse 0), program bitene kadar yaşar ve değerini çağrılar arasında korur; kapsamı yine yalnızca o bloktur.

H09_02_static_sayac.c · satır 10–17

```
10  void ziyaret(void)
11  {
12      int otomatik = 0;               /* her çağrıda yeniden oluşur ve 0 olur */
13      static int statik = 0;          /* program başında bir kez 0 olur */
14      otomatik++;
15      statik++;                       /* değeri çağrılar arasında korunur */
16      printf("%d. çağrı: otomatik = %d, statik = %d\n", statik, otomatik, statik);
17  }
```

ÇIKTI

```
$ ./static_sayac
1. çağrı: otomatik = 1, statik = 1
2. çağrı: otomatik = 1, statik = 2
3. çağrı: otomatik = 1, statik = 3
4. çağrı: otomatik = 1, statik = 4
```

`otomatik` her çağrıda 0'dan başlar, `statik` 1, 2, 3, 4 diye sayar: `static int statik = 0;` her çağrıda çalışan bir atama değil, program başında **bir kez** verilen ilk değerdir. `main`'de `statik = 0;` yazmak derlenmez: ömür program kadar, kapsam blok kadardır.

Tablo 9.1
Kapsam, ömür ve bellek bölgesi: dört bildirim biçimi

Bildirim yeri ve biçimi	Kapsam	Ömür	İlk değer verilmezse	Bellek bölgesi (3.6)
Fonksiyon içinde: `int x;`	blok	otomatik	belirsiz	yığın (stack)
Fonksiyon içinde: `static int x;`	blok	statik	0	data / bss
Fonksiyonların dışında: `int x;`	dosya (dış bağlantı)	statik	0	data / bss
Fonksiyonların dışında: `static int x;`	dosya (iç bağlantı)	statik	0	data / bss

3.4 Çok dosyalı programlar: .h, .c, extern, static ve başlık koruyucusu

Bir C **modülü** iki dosyadır: **başlık** (`.h`) **arayüzdür**, yalnızca **bildirimler** (prototipler, `extern`) içerir; **kaynak** (`.c`) **gerçeklemedir**: gövdeler ve bellek ayıran **tanımlar** (Kernighan & Ritchie, 1988, Bölüm 4; King, 2008, Bölüm 15). Kullanan dosya yalnızca başlığı `#include` eder. Örnek 4'ün başlığı:

H09_04_cok_dosyali/matematik_util.h

```c
/* matematik_util.h — Hafta 9 · Örnek 4 (★★): modülün ARAYÜZÜ
 * Yalnızca bildirimler: prototipler ve extern değişken bildirimi.
 * Tanımlar (gövdeler, bellek ayıran değişkenler) matematik_util.c'dedir.
 */
#ifndef MATEMATIK_UTIL_H           /* başlık koruyucusu (include guard) */
#define MATEMATIK_UTIL_H

extern const double PI_YAKLASIK;    /* bildirim: tanımı başka dosyada */

long long faktoriyel(int n);
int ebob(int a, int b);
double daire_alani(double r);
int cagri_sayisi(void);

#endif /* MATEMATIK_UTIL_H */
```

Önişlemci metin yapıştırır: `#include` satırı, başlığın metniyle değiştirilir (`H09_K04_ceviri_birimi.c` bunu elle yapar, çıktı aynıdır). Bir `.c` dosyasının başlıklarıyla oluşturduğu metne **çeviri birimi** (translation unit) denir; her biri **ayrı** derlenir. **Başlık koruyucusu** (`#ifndef` / `#define` / `#endif`) aynı başlığın bir çeviri biriminde ikinci kez içerilen kopyasını boşaltır: başlığı bilerek iki kez içeren `H09_K05_cift_icerme.c`'nin önişlemci çıktısında prototip koruyucuyla **1**, koruyucusuz kopyayla **2** kez geçti (gerçek).

Bağlantı: `extern` ve `static`. `extern const double PI_YAKLASIK;` bir **bildirimdir**: "ad var, tanımı başka dosyada"; tanım yalnızca `matematik_util.c`'de, **bir kez** yazılır. Dosya kapsamında `static` ise adı **iç bağlantılı** (internal linkage) yapar: yalnızca o `.c` görür. Modül iç durumunu böyle gizler ve yalnızca bir **erişim fonksiyonu** açar (`int cagri_sayisi(void) { return cagri; }`):

H09_04_cok_dosyali/matematik_util.c · satır 4–8

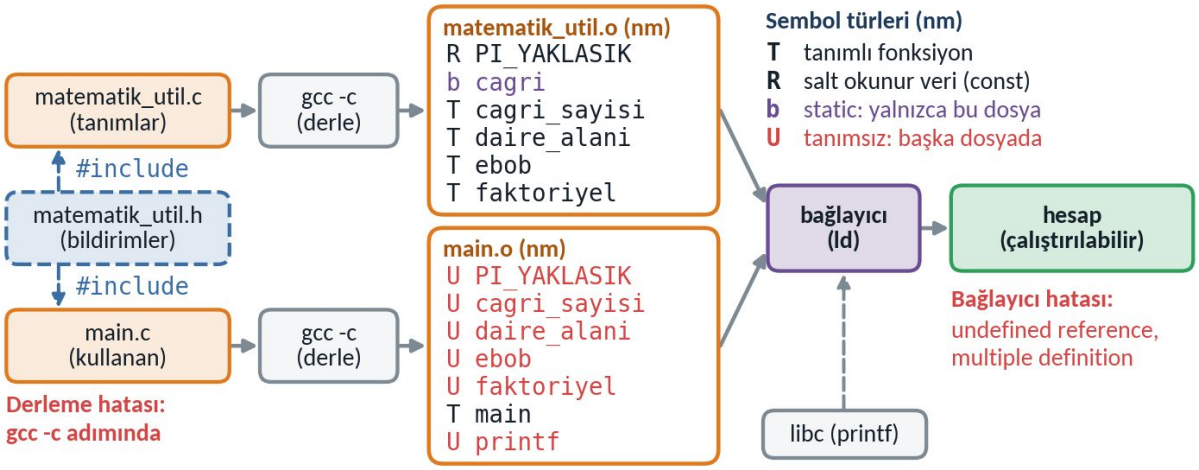
```c
#include "matematik_util.h"         /* kendi başlığı: bildirim–tanım uyumu */

const double PI_YAKLASIK = 3.14159265358979;  /* extern'in tanımı */

static int cagri = 0;               /* dosya kapsamı + static: iç bağlantı */
```

Derleme hattı: her `.c`, `gcc -c` ile ayrı derlenip **nesne dosyasına** (`.o`) dönüşür; **bağlayıcı** (linker, `ld`) nesne dosyalarını ve C kitaplığını birleştirir (Şekil 9.2). `nm`'nin sembol tablosunda `matematik_util.o`'nun fonksiyonları `T`, `PI_YAKLASIK R`, `static cagri` küçük `b`'dir (yerel sembol); `main.o`'da modülün adları `U`'dur (undefined) ve bağlayıcı her `U`'yu bir tanımla eşler.

Şekil 9.2
Çok dosyalı derleme hattı: #include, gcc -c, semboller ve bağlayıcı (Örnek 4'ün gerçek nm çıktısı)



Derleyici mi, bağlayıcı mı? `main.c`'ye `cagri = 0;` yazılırsa hata **derlemede** çıkar ("'cagri' undeclared (first use in this function)"). `H09_K06_static_erisim.c` ise `extern int cagri;` ile **derlenir**, ama **bağlamada** `static cagri` bulunamaz: bağlayıcı "undefined reference" iletisi verir (2 kez, `cagri`'nin geçtiği her satır için). `dosya.c:satır: error:` biçimi **derleyiciden**, `/usr/bin/ld: …` ve `collect2: error: ld returned 1 exit status` **bağlayıcıdan** gelir.

3.5 Önişlemci makroları ve tuzakları

`#define` ile tanımlanan **makro**, derlemeden önce yapılan **metin değiştirmedir**: önişlemci öncelik bilmez, yalnızca metni kopyalar.

H09_05_kare_makro.c · satır 11–17

```c
#define KARE_YANLIS(x) x * x        /* parantezsiz: tuzak! */
#define KARE(x) ((x) * (x))         /* her argüman ve bütün ifade parantezli */

int kare(int x)                     /* fonksiyon: argüman bir kez hesaplanır */
{
    return x * x;
}
```

`gcc -E` derleyicinin gördüğü metni verir: `KARE_YANLIS(1 + 2)` → `1 + 2 * 1 + 2` = **5**; `100 / KARE_YANLIS(5)` → `100 / 5 * 5` = **100**. Kurallar: her parametre ve bütün ifade parantezli: `((x) *`

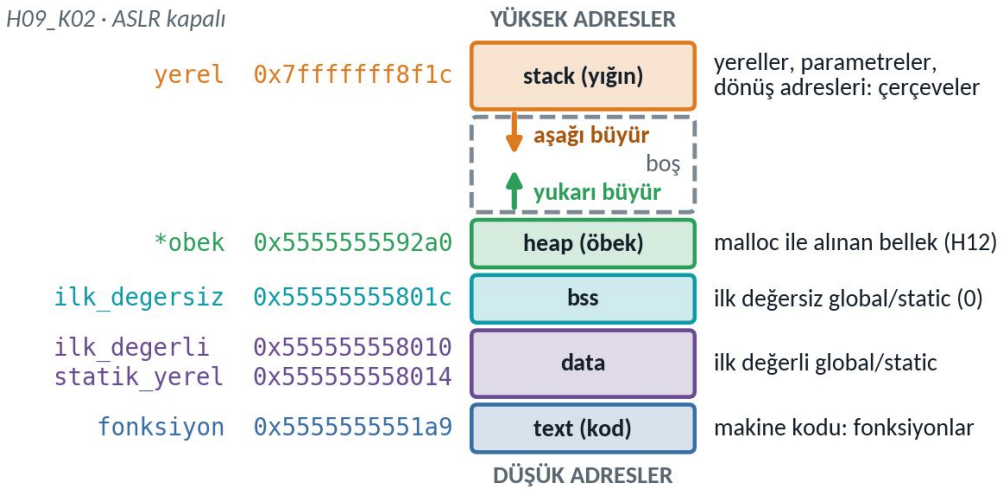
`(x))`. Gerçek çıktı: "KARE_YANLIS(1 + 2) = 5", "KARE(1 + 2) = 9". Makronun bir üstünlüğü tipten bağımsızlığıdır (`KARE(2.5)` → 6.25).

Parantez her şeyi çözmez: `KARE(i++)` → `((i++) * (i++))`; aynı ifadede `i`'yi sıralamasız iki kez değiştirmek **tanımsız davranıştır**, "şu değer çıkar" diye bir yanıt yoktur; GCC `-Wsequence-point` ile uyarır (HA6). Bu yüzden basit hesaplar için **fonksiyon** (argüman bir kez hesaplanır, tip denetlenir), sabitler için `const` tercih edilir.

3.6 Kaputun altında: çağrı yığını ve süreç bellek düzeni

Çalışan bir program bir **sanal adres alanı** alır; alan bölgelere ayrılır (Bryant & O'Hallaron, 2016): **text** (makine kodu), **data** (ilk değerli global/`static`), **bss** (ilk değersiz global/`static`, başta 0), **öbek** (heap; `malloc`, H12) ve **yığın** (stack; çağrı çerçeveleri). `H09_K02_bellek_duzeni.c` her bölgeden bir nesne tanımlar (`ilk_degerli` ve `ilk_degersiz` global, `statik_yerel` static yerel, `yerel` otomatik, `malloc` ile bir öbek nesnesi ve `fonksiyon`'un kendisi) ve adreslerini `%p` ile yazdırır (Şekil 9.3).

Şekil 9.3
Süreç bellek haritası: yüksek adresler üstte; ASLR kapalı çalıştırmanın gerçek adresleriyle (H09_K02)



> KAPUTUN ALTINDA · **ASLR ve taşınabilirlik**
>
> Şekil 9.3 ASLR kapalı bir çalıştırmanın (`setarch x86_64 -R`) adresleridir. Sıradan iki çalıştırmada `yerel` 0x7fff0b635f5c ve 0x7fffb62e45ec adresinde çıktı: Linux her çalıştırmada bölgeleri rastgele kaydırır (**ASLR**); bölge içi farklar aynı kalır. `nm fonksiyon`'u **T**, `ilk_degerli`'yi **D**, `ilk_degersiz`'i **B**, `statik_yerel.0`'ı **d** olarak listeler; `yerel` hiçbir tabloda yoktur. Fonksiyon adresini `(void *)` ile yazdırmayı ISO C garanti etmez (POSIX'te çalışır); `-pedantic` uyardığı için o satır `#pragma GCC diagnostic` ile işaretlidir.

Yığın çerçevesi ve dönüş adresi

Her çağrı yığında bir **yığın çerçevesi** (stack frame) açar: parametreler, yereller, kaydedilmiş yazmaçlar ve **dönüş adresi**. x86-64'te `call` dönüş adresini yığına iter (push) ve atlar; `ret` onu çeker (pop) ve geri döner. Yığın son giren–ilk çıkar (LIFO) düzenindedir, bu yüzden iç içe binlerce çağrıda bile dönüşler karışmaz; x86-64 Linux'ta **aşağı büyür**. `H09_K03_yigin_adresleri.c` özyinelemeli faktöriyelin her çerçevesindeki `&n`'yi ve bir öncekiyle farkını yazdırır (farkı hesaplamak için adres `uintptr_t`'ye çevrilir; adres aritmetiği H12'de).

Gözlemler (gerçek): her yeni çerçeve bir öncekinin **altında**; ASLR yüzünden ikinci çalıştırmada faktoriyel(4)'ün `&n`'i 0x7fff1f80535c çıktı ama fark yine **64 baytı** (-O0); `-O2` ile çerçeve **32 bayta** indi, çünkü iyileştirici yerelleri yazmaçlarda tutar. C standardı "yığın" sözcüğünü bile kullanmaz. Makine

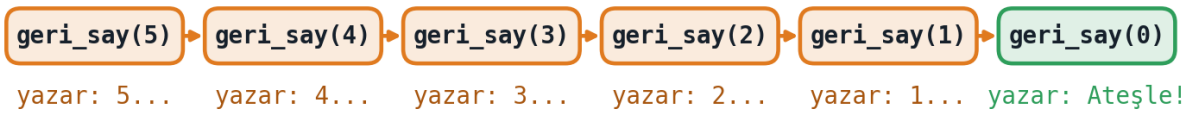
kodunda (`gcc -O0 -S kodlar/ciktilar/H09_K07_*_asm.txt`) `call faktoriyel` fonksiyonun **kendini** çağırdığını gösterir: özyineleme sıradan bir `call`'dır.

3.7 Özyineleme: problemi kendi küçük örneğine indirgemek

Özyinelemeli bir fonksiyon işinin bir parçası için **kendini** çağırır. Üç koşul gerekir: (1) **temel durum** (base case): kendini çağırmadan dönen en küçük örnek; (2) **özyinelemeli adım**: problemi aynı türden **daha küçük** bir probleme indirgeyen çağrı; (3) **temel duruma yaklaşma**: her çağrıda bir ölçü kesin küçülür. Örnek 3'teki geri sayım üçünü gösterir (Şekil 9.4). Anahtar bir **güven sıçramasıdır**: `geri_say(n - 1)`'in doğru çalıştığını varsayarız; temel durum doğru ve her adım küçülüyorsa, tümevarımdaki gibi her n için doğrudur (tuzaklar: HA1, HA2).

Şekil 9.4
Özyinelemeli fonksiyonun üç parçası (yeşil: temel durum, mavi: çerçevenin işi, turuncu: özyinelemeli adım) ve geri_say(5)'in çağrı zinciri (H09_03, gerçek çıktı)

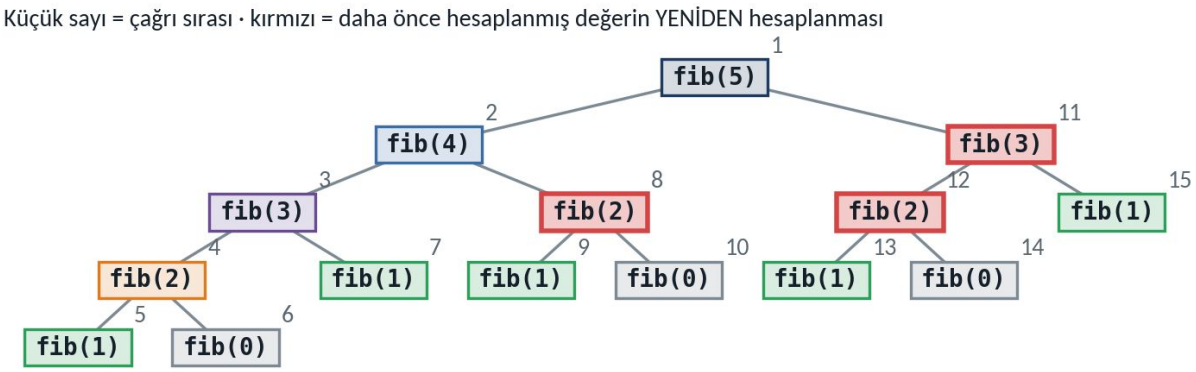
```
10  void geri_say(int n)
11  {
12      if (n == 0) {                   /* temel durum: artık çağrı yok */
13          printf("Ateşle!\n");
14          return;
15      }
16      printf("%d...\n", n);
17      geri_say(n - 1);                /* özyinelemeli adım: n küçülüyor */
18  }
```



Özyineleme ağacı

Fonksiyon kendini **birden çok kez** çağırıyorsa çağrılar bir **ağaç** oluşturur. fib(0) = 0, fib(1) = 1, fib(n) = fib(n − 1) + fib(n − 2) tanımı C'ye harfi harfine çevrilir; Şekil 9.5, `H09_K09_fib_agaci.c`'nin girintili çıktısından çizildi: fib(5) **15** çağrı yapar; fib(3) **2**, fib(2) **3** kez sıfırdan hesaplanır. n büyüdükçe tekrar patlar: fib(30) 2 692 537, fib(40) 331 160 281 çağrı (Örnek 12). Derinlik yalnızca n'dir: sorun **zamandır**.

Şekil 9.5
fib(5) özyineleme ağacı: 15 çağrı; kırmızı düğümler yeniden hesaplanan değerler (H09_K09 çıktısından)



Gerçek çıktı (H09_K09): Toplam 15 çağrı; fib(2) 3 kez hesaplandı.

Yığın taşması

Temel duruma hiç ulaşmayan özyineleme yığını doldurur: **yığın taşması** (stack overflow). Bu ortamda `ulimit -s` **8192 KiB** (8 MiB; Linux'ta tipik varsayılan); MSVC bağlayıcısı **1 MB** ayırır (Microsoft, t.y.), MinGW'nin GNU ld'si PE hedeflerinde 2 MB (Free Software Foundation, t.y.-b). Örnek 11'de hedef -1 girilince temel duruma ulaşılmaz:

ÖRNEK 11 (GERÇEK, KESİT)

```
$ ./yigin_tasmasi; echo "çıkış durumu: $?"      # hedef -1
Hedef derinlik (-1: hiç ulaşılamaz): -1

derinlik 10000
…
derinlik 260000
bash: line 1:  8849 Segmentation fault      ./yigin_tasmasi
çıkış durumu: 139
```

Program yaklaşık 260 000 derinlikte **Segmentation fault** ile çöktü; çıkış durumu **139** (128 + 11, SIGSEGV). Yığın sınırını değiştirerek (rapor her 1000 çağrıda) ölçtük: 8 192 KiB → 261 000, 4 096 KiB → 130 000, 2 048 KiB → 64 000, 1 024 KiB → 31 000. Derinlik yığın boyuyla **orantılıdır**: 8 MiB / 261 000 ≈ 32,1 bayt, `dal()`'ın -O0 çerçevesine eşit (dönüş adresi 8 + `rbp` 8 + `sub rsp, 16`). AddressSanitizer çöküşü adıyla raporlar: "ERROR: AddressSanitizer: stack-overflow".

3.8 Yinelemeli ↔ özyinelemeli dönüşüm

Her döngü özyinelemeyle, her özyineleme de (gerekirse açık bir yığınla) döngüyle yazılabilir. Tarif: döngünün **bitiş koşulu** temel duruma, **biriktirici** dönüş değerindeki işleme (`n * …`), **güncelleme** çağrının argümanına (`n - 1`) dönüşür (`H09_K07_faktoriyel_saf.c`):

H09_K07_faktoriyel_saf.c · satır 11–26

```
11  long long faktoriyel_dongu(int n)   /* yinelemeli: döngü + biriktirici */
12  {
13      long long sonuc = 1;
14      for (int i = 2; i <= n; i++) {
15          sonuc *= i;
16      }
17      return sonuc;
18  }
19
20  long long faktoriyel(int n)         /* özyinelemeli: tanımın kendisi */
21  {
22      if (n <= 1) {                   /* temel durum: 0! = 1! = 1 */
23          return 1;
24      }
25      return n * faktoriyel(n - 1);   /* n! = n · (n-1)! */
26  }
```

Tablo 9.2
Yinelemeli ve özyinelemeli çözümlerin karşılaştırılması

Ölçüt	Yinelemeli (döngü)	Özyinelemeli
Bellek	Sabit: birkaç yerel değişken	Derinlik × çerçeve boyu (32–64 bayt/çağrı ölçtük)
Zaman	Çağrı maliyeti yok	Çerçeve aç/kapa; fib'de tekrarlanan hesap
Sınır	Yalnızca tip sınırı	Yığın boyu: Linux 8 MiB, Windows (MSVC) 1 MB
Uygun olduğu yer	Doğrusal tarama, sayaç, biriktirme	Problem doğal olarak küçük kopyalara bölünüyorsa (Öklid, Hanoi)

> MERAK KUTUSU · **Kuyruk özyinelemesi: derleyici özyinelemeyi döngüye çevirebilir mi?**
>
> Özyinelemeli çağrı fonksiyondaki **son iş** ise buna **kuyruk özyinelemesi** (tail recursion) denir: `H09_K08_kuyruk_ozyineleme.c`'deki `return faktoriyel_kuyruk(n - 1, birikim * n);` ve Örnek 10'daki `ebob(b, a % b)` böyledir. Gözlemimiz (gerçek assembly, `kodlar/ciktilar/H09_K08_*_asm.txt`): `gcc -O0` her iki faktöriyelde `call` üretti; `gcc -O2` ise kuyruk özyinelemeli sürümü **ve** sıradan `n * faktoriyel(n - 1)` sürümünü `call` içermeyen döngülere (`jne`) çevirdi. Ama bu bir **iyileştirmedir, güvence değil**: C standardı kuyruk çağrısı iyileştirmesini garanti etmez; `-O0` ya da başka bir derleyici aynı kodu yığını tüketen özyineleme olarak bırakabilir.

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır (`kodlar/H09_01_…` – `H09_13_…`); hepsi `-Wall -Wextra -pedantic` ile **sıfır uyarıyla** derlenir, bilerek çöken Örnek 11 ve süre ölçen Örnek 12 dışındakiler ASan + UBSan ile de temizdir; çıktılar gerçektir. Örnek 6 ve 9 standart çözümlü örnek şablonunun tamamıyla işlenir.

4.1 ★ basamaklar: kapsam, static ve ilk özyineleme (Örnek 1–3)

Örnek 1 (kapsam) ve **Örnek 2** (static sayaç) Bölüm 3.1 ve 3.3'te incelendi. **Değiştir.** (a) Örnek 1'de `int x = 3;` satırını `x = 3;` yapın: "bloktan sonra" ne yazar? (b) Örnek 2'de `static`'i silin: dördüncü satır ne olur? **Örnek 3 ★ — geri sayım** (kod Şekil 9.4'de):

ÇIKTI
```
$ ./geri_sayim
5...
4...
3...
2...
1...
Ateşle!
```

Değiştir. `printf` ile özyinelemeli çağrının yerini değiştirin: çıktı neden 1, 2, 3, 4, 5 olur (Örnek 8)? `geri_say(-3)` temel duruma yaklaşır mı?

4.2 Örnek 4 ★★ — Çok dosyalı proje: matematik_util

Problem. Faktöriyel, EBOB ve daire alanını yeniden kullanılabilir bir **modüle** taşıyın; modül kendi çağrı sayısını tutsun ama sayı dışarıdan değiştirilemesin. Arayüz ve iç durum Bölüm 3.4'te; kullanan program:

H09_04_cok_dosyali/main.c · satır 8–19

```c
#include "matematik_util.h"

int main(void)
{
    printf("10! = %lld\n", faktoriyel(10));
    printf("EBOB(84, 36) = %d\n", ebob(84, 36));
    printf("Yarıçapı 2 olan dairenin alanı = %.4f\n", daire_alani(2.0));
    printf("PI_YAKLASIK = %.5f (extern ile)\n", PI_YAKLASIK);
    printf("Modül fonksiyonları %d kez çağrıldı.\n", cagri_sayisi());
    /* cagri = 0;  → derlenmez: cagri, matematik_util.c'de static */
    return 0;
}
```

AYRI DERLEME, BAĞLAMA VE ÇALIŞTIRMA (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c matematik_util.c
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c main.c
$ gcc matematik_util.o main.o -o hesap
$ ./hesap
10! = 3628800
EBOB(84, 36) = 12
Yarıçapı 2 olan dairenin alanı = 12.5664
PI_YAKLASIK = 3.14159 (extern ile)
Modül fonksiyonları 3 kez çağrıldı.
```

`main.c` sayacı **okuyabilir** (`cagri_sayisi()`), **değiştiremez**. Tek komut da olur (`gcc -std=c17 -Wall -Wextra -pedantic -g matematik_util.c main.c -o hesap`; GCC dosyaları yine ayrı derleyip bağlar); ayrı derlemenin kazancı yalnızca değişen dosyanın yeniden derlenmesidir (Mini-OBS'de `make`, Bölüm 7). Windows/MinGW'de komutlar aynıdır, çıktı `hesap.exe` olur. **Değiştir.** Modüle `us_al` ekleyin: hangi dosyalar değişir? `main.c`'den `#include`'u silin: ne çıkar?

4.3 Örnek 5 ★★ — KARE(x) makro tuzağı

Kod ve `gcc -E` açılımı Bölüm 3.5'tedir. **Değiştir.** `#define TOPLA(a, b) a + b` ile `2 * TOPLA(3, 4)`'ü önce kâğıtta, sonra `gcc -E` ile hesaplayın; `KARE(i++)`'nın `-Wall` iletisini okuyun (HA6).

4.4 Amiral örnek — Örnek 6 ★★: özyinelemeli faktöriyel ve yığın çizimi

1. Problem. n! değerini tanımın kendisiyle (n! = n · (n − 1)!) özyinelemeli hesaplayın ve her çağrının başlayıp bittiği anı ekranda izleyerek çağrı yığınının dolup boşalmasını görün.

GİRDİ	İŞLEM	ÇIKTI
• n (0 ≤ n ≤ 20)	• Temel durum: n ≤ 1 ise 1 • Adım: n · faktoriyel(n − 1) • Girişte ve dönüşte izleme satırı	• n! • Çerçevelerin açılış/kapanış sırası

2–3. Çözümleme ve algoritma. 20!, `long long`'a sığan son faktöriyeldir (H6); 21! işaretli taşmaya (tanımsız davranış) yol açar. `n <= 1` temel durumu 0!, 1! ve negatif n'yi de kapsar. İzleme için dosya kapsamında `static int derinlik`: girişte artar (push), dönüşte azalır (pop).

4. C kodu.

H09_06_faktoriyel_yigin.c · satır 9–34

```c
#include <stdio.h>

static int derinlik = 0;            /* izleme için: açık çerçeve sayısı */

long long faktoriyel(int n)
{
    derinlik++;                     /* PUSH: yeni çerçeve */
    printf("%*s→ faktoriyel(%d) çağrıldı   [derinlik %d]\n",
           2 * derinlik, "", n, derinlik);
    long long sonuc;
    if (n <= 1) {
        sonuc = 1;                  /* temel durum */
    } else {
        sonuc = n * faktoriyel(n - 1);  /* özyinelemeli adım */
    }
    printf("%*s← faktoriyel(%d) = %lld döndü\n", 2 * derinlik, "", n, sonuc);
    derinlik--;                     /* POP: çerçeve kapanır */
    return sonuc;
}

int main(void)
{
    long long f = faktoriyel(4);
    printf("4! = %lld\n", f);
    return 0;
}
```

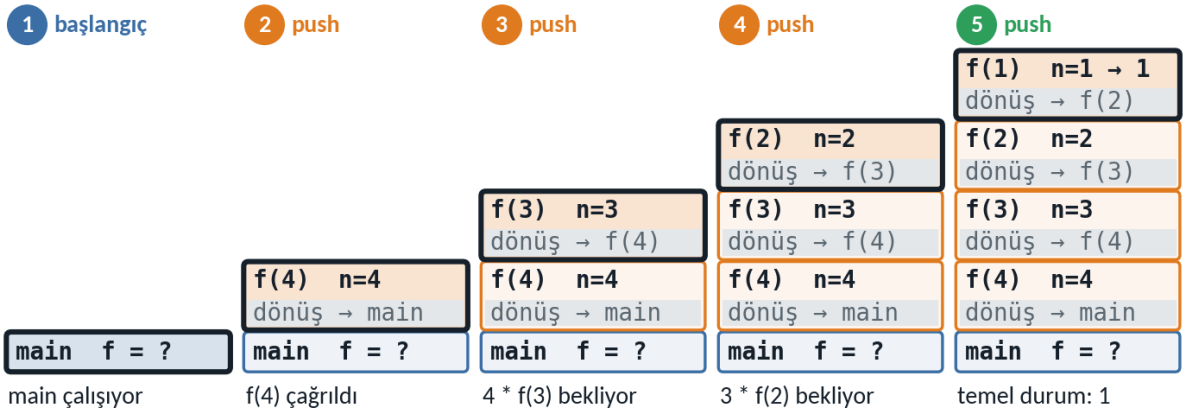
5. Satır satır. Satır 11: bütün çağrılar **aynı** sayacı paylaşır. Satır 16: `%*s`, `2 * derinlik` genişliğinde boşluk yazar; girinti derinliği gösterir. Satır 22: çarpma ancak `faktoriyel(n - 1)` döndükten sonra yapılabilir; çerçeve bu yüzden **bekler**. Her çerçevenin **kendi** `n`'si ve `sonuc`'u vardır: dört `n` aynı anda yığındadır.

6. İzleme. Her satır bir push ya da pop olayıdır; "bekleyen iş", dönüşte yapılacak çarpmadır (push: Şekil 9.6):

Tablo 9.3
faktoriyel(4) push/pop izleme tablosu (çerçeve sayısı main dahil, olaydan sonra; dönen değerler gerçek çıktıdan)

Adım	Olay	Çağrı / dönüş	Çerçeve sayısı (olaydan sonra)	Bekleyen iş	Dönen değer
1	push	main → f(4)	2	4 * f(3)	–
2	push	f(4) → f(3)	3	3 * f(2)	–
3	push	f(3) → f(2)	4	2 * f(1)	–
4	push	f(2) → f(1)	5	**temel durum**	–
5	pop	f(1) → f(2)	4	–	1
6	pop	f(2) → f(3)	3	2 * 1	2
7	pop	f(3) → f(4)	2	3 * 2	6
8	pop	f(4) → main	1	4 * 6	**24**

Şekil 9.6
faktoriyel(4) (f): çerçeveler sırayla açılıyor (push); etkin çerçeve koyu kenarlı, gri satır dönüş adresi



7. Örnek çalıştırma (gerçek):

```
ÇIKTI
$ ./faktoriyel_yigin
  → faktoriyel(4) çağrıldı   [derinlik 1]
    → faktoriyel(3) çağrıldı   [derinlik 2]
      → faktoriyel(2) çağrıldı   [derinlik 3]
        → faktoriyel(1) çağrıldı   [derinlik 4]
        ← faktoriyel(1) = 1 döndü
      ← faktoriyel(2) = 2 döndü
    ← faktoriyel(3) = 6 döndü
  ← faktoriyel(4) = 24 döndü
4! = 24
```

8. Test. İzlemesiz saf sürüm `H09_K07_faktoriyel_saf.c` yinelemeliyle karşılaştırıldı; sınır (0!), normal (5!) ve üst sınır (20!) aynı: "0! = 1 = 1", "5! = 120 = 120", "20! = 2432902008176640000 = 2432902008176640000". **9. Değiştir.** `faktoriyel(21)`'i `-fsanitize=undefined` ile çalıştırın; çarpmayı çağrıdan önce yapan bir **birikim** parametresi ekleyin (Bölüm 3.8): "bekleyen iş" ne olur?

4.5 Örnek 7 ★★ — 1..N toplamı ve üs alma

H09_07_toplam_us.c · satır 11–24

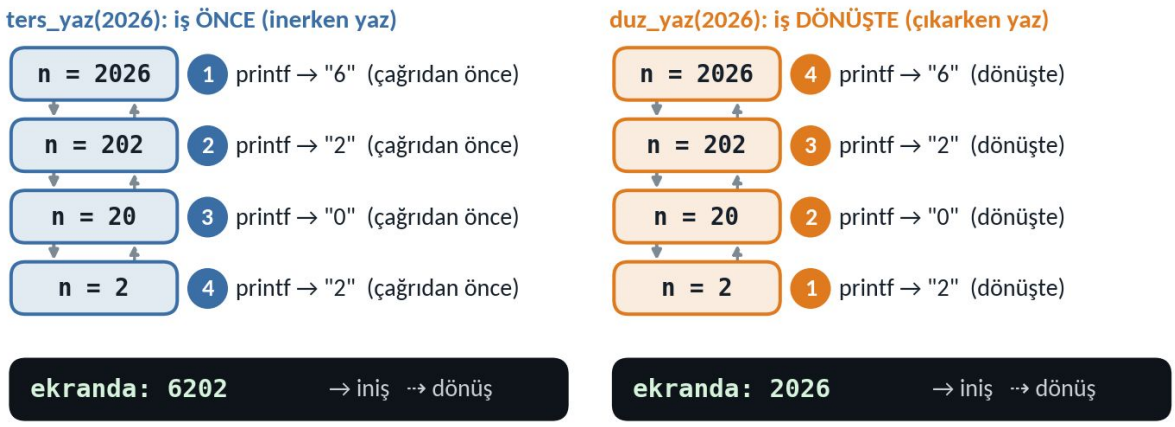
```
11  long long toplam(int n)
12  {
13      if (n == 0) {                   /* temel durum: boş toplam */
14          return 0;
15      }
16      return n + toplam(n - 1);       /* n + (1..n-1 toplamı) */
17  }
18
19  long long us_al(long long taban, int us)
20  {
21      if (us == 0) {                  /* temel durum: x^0 = 1 */
22          return 1;
23      }
24      return taban * us_al(taban, us - 1);
```

İki fonksiyon da problemi **bir küçüğüne** indirger; temel durumlar boş toplam (0) ve $x^0 = 1$'dir: H6'daki biriktiricilerin **başlangıç değerleri**. Gerçek çıktılardan: "toplam(100) = 5050 (formül: 5050)", "us_al(2, 10) = 1024", "us_al(-2, 5) = -32", "us_al(7, 0) = 1". `toplam(100)` 101 çerçeve açar; `toplam(1000000)` yığına sığmayabilir, oysa Gauss formülü (H2) tek işlemdir. Formül varsa formül, yoksa döngü; özyineleme, problemin yapısı istediğinde.

4.6 Örnek 8 ★★ — Basamak toplamı ve "dönüşte iş yapma"

`basamak_toplami(n)` = son basamak + `basamak_toplami(n / 10)` (kod, HA7'nin düzeltilmiş sürümüyle aynı). Yenilik, aynı dosyadaki iki yazdırma fonksiyonundadır:

Şekil 9.7
İş önce (ters_yaz) ve iş dönüşte (duz_yaz): aynı çağrı zinciri, farklı yazdırma sırası (H09_08 gerçek çıktısıyla)



H09_08_basamak_ters.c · satır 19–33

```
19  void ters_yaz(int n)                /* İŞ ÖNCE: son basamak önce yazılır */
20  {
21      printf("%d", n % 10);
22      if (n >= 10) {
23          ters_yaz(n / 10);
24      }
25  }
26
27  void duz_yaz(int n)                 /* İŞ DÖNÜŞTE: ilk basamak önce yazılır */
28  {
29      if (n >= 10) {
30          duz_yaz(n / 10);
31      }
32      printf("%d", n % 10);
33  }
```

ÇIKTI

```
$ ./basamak_ters
basamak_toplami(2026) = 10
ters_yaz(2026): 6202
duz_yaz(2026):  2026
ters_yaz(1230): 0321
```

İki fonksiyon **aynı iki deyimi** içerir; yalnızca sıra farklıdır. `ters_yaz` yazdırmayı **inişte** yapar (son basamak önce); `duz_yaz` önce çağırır, basamağını **dönüşte** yazar: en derindeki çerçeve (ilk basamak) ilk döner (Şekil 9.7). Özyinelemenin en güçlü fikri: **çağrıdan sonra yapılan iş ters sırada yapılır**; yığın sırayı bizim yerimize çevirir. `ters_yaz(1230)` "0321" yazar: H6'daki sayısal ters çevirme 321 veriyordu; burada basamaklar metin olarak yazıldığı için 0 kaybolmaz.

4.7 Amiral örnek — Örnek 9 ★★: onluk sayıyı ikili yazdırma

1. Problem. Negatif olmayan bir tamsayının ikili gösterimini **doğru sırayla** yazdırın. H7'deki `H07_10_onluk_ikili.c` ardışık bölmeyle bitleri **ters sırada** yazmıştı (ilk üretilen kalan en düşük bittir); sorunu bilerek H9 (özyineleme) ve H10'a (dizi) bırakmıştık.

GİRDİ	İŞLEM	ÇIKTI
• n ≥ 0 (unsigned int)	• Temel durum: n < 2 → yalnızca n % 2 yazılır • Adım: önce ikili_yaz(n / 2), dönüşte n % 2	• n'nin ikili gösterimi (en anlamlı bit önce)

2–3. Çözümleme ve algoritma. n'nin ikili gösterimi = (n / 2)'nin ikili gösterimi + sonuna eklenen n % 2. Son bit **en sonda** yazılmalı: önce üst bitler (küçük problem), sonra kendi biti. `ikili_yaz(n): n ≥ 2 ise ikili_yaz(n / 2); n % 2'yi yaz.`

4. C kodu.

H09_09_ikili_yazdir.c · satır 10–24

```
10  void kalanlari_yaz(unsigned int n)  /* H07_10'un yöntemi: ardışık bölme */
11  {
12      do {
13          printf("%u", n % 2);        /* en düşük bit ÖNCE çıkar */
14          n /= 2;
15      } while (n > 0);
16  }
17
18  void ikili_yaz(unsigned int n)      /* özyinelemeli çözüm */
19  {
20      if (n >= 2) {
21          ikili_yaz(n / 2);           /* önce üst bitler */
22      }
23      printf("%u", n % 2);            /* dönüşte bu çerçevenin biti */
24  }
```

5. Satır satır. `kalanlari_yaz` H07_10'un yöntemidir (karşılaştırma için). Satır 20: n < 2 ise çağrı yapılmaz (temel durum). Satır 21: n her çağrıda yarıya iner; 32 bitlik `unsigned int` için en çok 32 çerçeve. Satır 23: her çerçevede **dönüşte** çalışır (`%u`: `unsigned int` belirteci).

Tablo 9.4
ikili_yaz(13) izleme tablosu: çerçeveler 1'den 4'e açılır, bitler 4'ten 1'e (dönüş sırasıyla) yazılır

Çerçeve	n	n ≥ 2 ?	çağrı	dönüşte yazılan	yazma sırası
1	13	evet	ikili_yaz(6)	13 % 2 = 1	4.
2	6	evet	ikili_yaz(3)	6 % 2 = 0	3.
3	3	evet	ikili_yaz(1)	3 % 2 = 1	2.
4	1	hayır	–	**1 % 2 = 1**	1.

7–8. Örnek çalıştırma ve test (gerçek). İlk satır H07_10 yöntemiyle karşılaştırmadır: "Kalanlar üretildikleri sırayla (H07_10): 1011 <- ters!". Testler beklenen değerleri verdi: sınır "0 = 0", "1 = 1"; ikinin kuvveti "4 = 100"; amiral girdi "13 = 1101 (ikili)"; büyük sayı "2026 = 11111101010" ✓.

9. Değiştir. (a) `taban_yaz(n, taban)`: 2 ≤ taban ≤ 10 (8 tabanında 2026 → 3752). (b) 16 tabanında `A`–`F`. (c) Bitleri yazdırmak yerine sayın (Alıştırma 9.5). (d) Döngü + **dizi** ile aynı iş H10'da.

4.8 Örnek 10 ★★ — Özyinelemeli EBOB (Öklid)

H09_10_ebob_ozyinelemeli.c · satır 11–18

```
11  int ebob(int a, int b)
12  {
13      printf("  ebob(%d, %d)\n", a, b);  /* her çağrıyı göster */
14      if (b == 0) {                    /* temel durum */
15          return a;
16      }
17      return ebob(b, a % b);           /* özyinelemeli adım: b kesin küçülür */
18  }
```

Öklid, tanımıyla yazılınca matematik metnine dönüşür: ebob(a, 0) = a, ebob(a, b) = ebob(b, a mod b). Gerçek zincir: ebob(48, 18) → ebob(18, 12) → ebob(12, 6) → ebob(6, 0) → EBOB(48, 18) = 6. `b` her adımda kesin küçülür, temel duruma mutlaka ulaşılır; 18, 48 sırası ilk çağrıda kendiliğinden düzelir. H6 (`H06_11_ebob_ekok.c`) ve H8 (`H08_07_ebob_ekok.c`) aynı adımları döngüyle yaptı; buradaki çağrı son iş olduğu için kuyruk özyinelemesidir.

4.9 Örnek 11 ★★ — Sonsuz özyineleme ve yığın taşması

H09_11_yigin_tasmasi.c · satır 17–27

```
17  long dal(long derinlik, long hedef)
18  {
19      if (derinlik == hedef) {        /* temel durum: hedef derinlik */
20          return derinlik;
21      }
22      if (derinlik % ARALIK == 0) {
23          printf("derinlik %ld\n", derinlik);
24          fflush(stdout);             /* çökmeden önce çıktı kaybolmasın */
25      }
26      return dal(derinlik + 1, hedef);
27  }
```

Temel durum vardır ama hedef -1 ise **hiç sağlanmaz** (ölçümler Bölüm 3.7'de); 50 000 girilince "Temel duruma ulaşıldı: 50000 çağrı". `fflush(stdout)` gerekir: yönlendirilen çıktı tamponlanır, çöken program tamponu boşaltamaz. `-O2` kullanmayın: kuyruk özyinelemeli `dal` döngüye çevrilebilir. `ARALIK` `#ifndef` ile tanımlıdır, `-DARALIK=1000` ile değişir.

4.10 Örnek 12 ★★★ — Özyinelemeli Fibonacci ve çağrı sayısı

H09_12_fibonacci_cagri.c · satır 12–21

```c
static long long cagri = 0;         /* fib kaç kez çağrıldı? */

long long fib(int n)                /* özyinelemeli: tanımın birebir kopyası */
{
    cagri++;
    if (n < 2) {                    /* iki temel durum: fib(0) = 0, fib(1) = 1 */
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}
```

ÇIKTI

```
$ ./fibonacci_cagri
  n     fib(n)   çağrı sayısı  özyinelemeli       döngü  sonuç
  5          5             15        0.0 ms    0.000 ms  aynı
 10         55            177        0.0 ms    0.001 ms  aynı
 20       6765          21891        0.1 ms    0.000 ms  aynı
 30     832040        2692537       12.2 ms    0.001 ms  aynı
 35    9227465       29860703       89.2 ms    0.001 ms  aynı
 40  102334155      331160281     1032.5 ms    0.000 ms  aynı
```

Süreler makineye göre değişir, **çağrı sayıları değişmez**. fib(40): 331 160 281 çağrı ve yaklaşık 1 033 ms; yinelemeli `fib_dongu` (H7, `H07_09_fibonacci.c`) 40 turda, anında bitti. n her 5 arttığında çağrılar ≈ **11,1** katına çıkıyor (**üstel** büyüme); nedeni ağaçta (Şekil 9.5): aynı alt problemler **sıfırdan** yinelenir.

4.11 Örnek 13 ★★★ — Hanoi Kuleleri

Problem. n diski A'dan C'ye taşıyıp hamleleri sayın (B yardımcı). **Fikir:** (1) n − 1 diski B'ye, (2) en büyüğü C'ye, (3) n − 1 diski B'den C'ye; (1) ve (3) **aynı problemin küçüğüdür**, temel durum 0 disk (**böl ve fethet**).

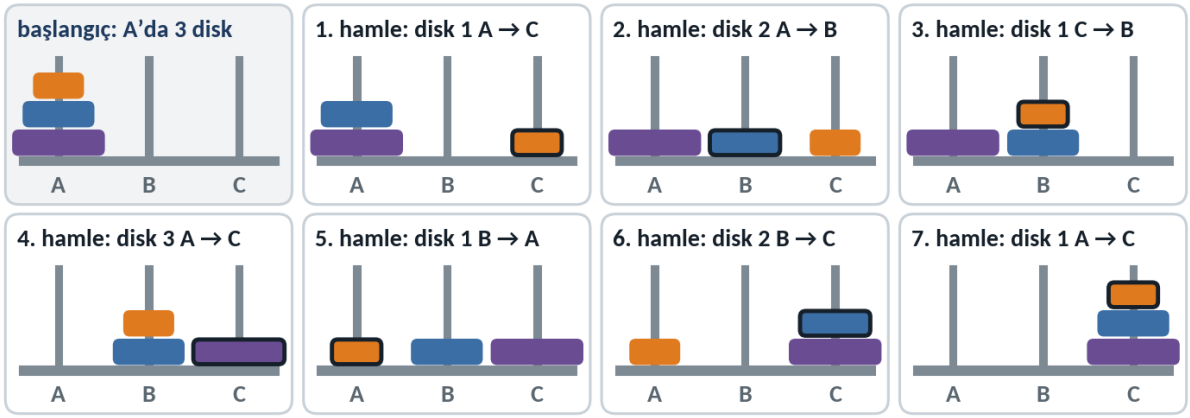
H09_13_hanoi.c · satır 12–23

```c
void hanoi(int n, char kaynak, char hedef, char yardimci, int yaz)
{
    if (n == 0) {                   /* temel durum: taşınacak disk yok */
        return;
    }
    hanoi(n - 1, kaynak, yardimci, hedef, yaz);   /* 1) üstteki n-1 disk */
    hamle++;                                      /* 2) en büyük disk */
    if (yaz) {
        printf("%2lld. hamle: disk %d  %c -> %c\n", hamle, n, kaynak, hedef);
    }
    hanoi(n - 1, yardimci, hedef, kaynak, yaz);   /* 3) n-1 disk üstüne */
}
```

GERÇEK ÇIKTI (KESİT)

```
$ ./hanoi      # ilk 7 satır: 3 diskin hamleleri (aşağıdaki şekil)
…
 n  sayılan hamle    2^n - 1
 1              1          1
 2              3          3
 3              7          7
 4             15         15
…
15          32767      32767
20        1048575    1048575
```

Şekil 9.8
3 disk için 7 hamle (H09_13'ün gerçek hamle listesinden çizildi; kalın kenarlı disk o hamlede taşınan)



Neden $2^n - 1$? Sayılan hamleler her n için $2^n - 1$ ile **aynı** çıktı. Gerekçe yapıdadır: H(n) = H(n − 1) + 1 + H(n − 1); bir disk eklemek hamleyi **iki katına çıkarıp bir ekler**: 1, 3, 7, 15, 31, … = $2^n - 1$. `H09_K10_hanoi_64.c` bu kuralı 64 kez uygulayıp `unsigned long long`'un tam sınırına ulaşır: 18 446 744 073 709 551 615 = $2^{64} - 1$ = `ULLONG_MAX`. Yığın derinliği ise yalnızca n + 1'dir: sorun bellek değil **zamandır** (Bölüm 1).

5. Hata Avcısı

Dört aile: **özyineleme** (temel durum yok ya da ulaşılamıyor; dönüş değeri kayboluyor), **kapsam** (global yan etki), **bağlama** (başlıkta tanım), **önişlemci** (makro). Dosyalar düzeltilmiş sürümleriyle `kodlar/hata_avcisi/` altındadır; iletiler ve çıktılar gerçektir (kabuktaki sayı süreç numarasıdır).

HA1 · Temel durum yok ve HA2 · Temel duruma ulaşılamıyor

`hata_avcisi/H09_HA1_temel_durum_yok.c · satır 10–13`

```
10  long long faktoriyel(int n)
11  {
12      return n * faktoriyel(n - 1);   /* temel durum unutuldu! */
13  }
```

HA1: DERLEYİCİ UYARISI (İLK SATIR) VE ÇALIŞTIRMA, GERÇEK

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H09_HA1_temel_durum_yok.c -o ha1
H09_HA1_temel_durum_yok.c:10:11: warning: infinite recursion detected [-Winfinite-
recursion]
…
$ ./ha1; echo "çıkış durumu: $?"
bash: line 1:  9420 Segmentation fault      ./ha1
çıkış durumu: 139
```

`-Wall` içindeki `-Winfinite-recursion` (Free Software Foundation, t.y.-a) dönmeyen bu özyinelemeyi yakalar. HA2 daha sinsidir: temel durum **vardır**, derleyici uyarmaz, çift n'de doğru çalışır:

hata_avcisi/H09_HA2_atlanan_temel_durum.c · satır 11–17

```
11  int azalan_toplam(int n)
12  {
13      if (n == 0) {                   /* yalnızca çift n bu duruma iner */
14          return 0;
15      }
16      return n + azalan_toplam(n - 2);
17  }
```

Gerçek çalıştırma: "azalan_toplam(10) = 30" doğru, ardından "Segmentation fault", çıkış durumu 139. `azalan_toplam(7)` 7 → 5 → 3 → 1 → −1 → … ile 0'ın **üzerinden atlar** ve yığın taşar. Düzeltme: **aralık** sınayan temel durum, `if (n <= 0)` ("azalan_toplam(7) = 16"). Adım 1'den büyükse ya da ölçü negatife inebiliyorsa temel durumu `==` ile değil `<=` ile yazın.

HA3 · Global değişkenin beklenmedik yan etkisi

hata_avcisi/H09_HA3_global_sayac.c · satır 10–18

```
10  int i;                              /* global sayaç: herkes değiştirebilir */
11
12  void cizgi_ciz(void)
13  {
14      for (i = 0; i < 20; i++) {
15          printf("-");
16      }
17      printf("\n");
18  }
```

`main` aynı `i` ile `for (i = 1; i <= 3; i++)` döngüsünde "Öğrenci i" yazıp `cizgi_ciz()`'i çağırır. Üç satır yerine yalnızca "Öğrenci 1" çıkar: `cizgi_ciz` `i`'yi 20 yapar, `i++` 21 yapar, `21 <= 3` yanlıştır. Derleyici bir şey söylemez. Düzeltme: her döngü kendi yerel `i`'sini `for` başlığında bildirir; fonksiyonlar arası veri parametre ve dönüş değeriyle taşınır.

HA4 · Başlık dosyasına tanım: "multiple definition"

`hata_avcisi/H09_HA4_cok_tanim/sayac.h · satır 5–12`

```c
#ifndef SAYAC_H
#define SAYAC_H

int toplam_cagri = 0;              /* HATA: tanım başlıkta! */

void say(void);

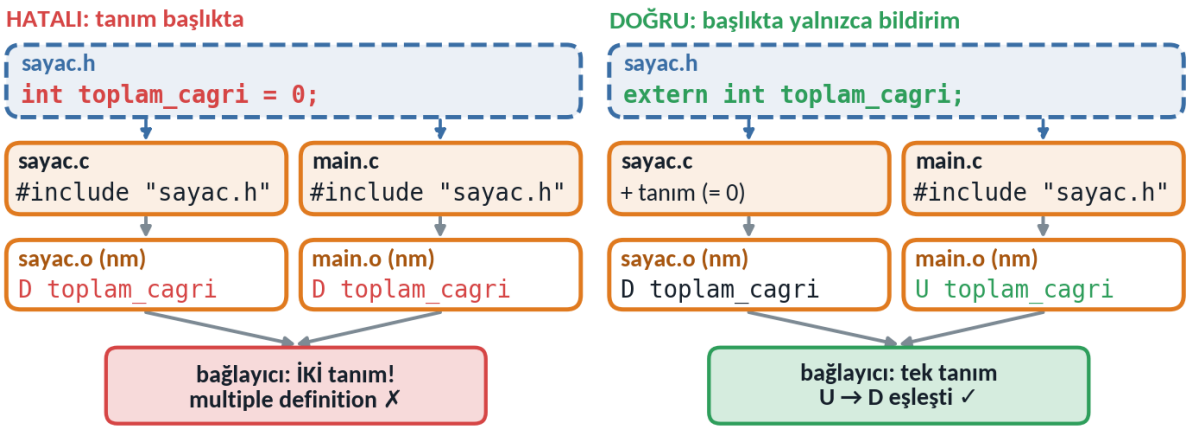
#endif /* SAYAC_H */
```

HA4: İKİ DOSYA DERLENİR, BAĞLAMA BAŞARISIZ OLUR (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c sayac.c
$ gcc -std=c17 -Wall -Wextra -pedantic -g -c main.c
$ gcc sayac.o main.o -o sayac
/usr/bin/ld: main.o:sayac.h:8: multiple definition of `toplam_cagri';
sayac.o:sayac.h:8: first defined here
collect2: error: ld returned 1 exit status
```

İki dosya da **ayrı ayrı sorunsuz derlenir**; ama `#include` metni yapıştırdığı için iki nesne dosyasında da `toplam_cagri`'nin birer **tanımı** vardır ve bağlayıcı birini seçemez. Koruyucu işe yaramaz: yalnızca **aynı çeviri birimindeki** ikinci kopyayı engeller (tek birimde birleştirilmiş `H09_HA4_tek_birim.c` sorunsuz çalışır). Tanım ilk değersiz olsa (`int toplam_cagri;`) da sonuç aynıdır: GCC 10'dan beri varsayılan `-fno-common` nedeniyle birden çok geçici tanım bağlayıcı hatasıdır (Free Software Foundation, 2020); `-fcommon` ile eski davranış döner (gerçek deneme `hata_avcisi/ciktilar/`). **Düzeltme:** başlıkta `extern int toplam_cagri;`, tanım yalnızca `sayac.c`'de (Şekil 9.9).

Şekil 9.9
HA4: başlıktaki tanım iki nesne dosyasında birer D üretir; extern bildirimle main.o'da U, sayac.o'da tek D



HA5 · Makroda parantez eksikliği ve HA6 · yan etkili argüman

`H09_HA5_makro_parantez.c`, `#define FARK(a, b) a - b` makrosunu `FARK(10, 3 - 1)` ve `2 * FARK(10, 4)` ile kullanır:

HA5: ÇALIŞTIRMA VE ÖNİŞLEMCİ ÇIKTISI (GERÇEK)

```
$ ./ha5
FARK(10, 3 - 1) = 6  (beklenen 8)
2 * FARK(10, 4) = 16  (beklenen 12)
$ gcc -E H09_HA5_makro_parantez.c | tail -n 9
    printf("FARK(10, 3 - 1) = %d  (beklenen 8)\n", 10 - 3 - 1);
    printf("2 * FARK(10, 4) = %d  (beklenen 12)\n", 2 * 10 - 4);
```

`10 - 3 - 1` = 6 ve `2 * 10 - 4` = 16: ifadeler geçerli C olduğu için derleyici uyarmaz. `((a) - (b))` ile düzeltilmiş sürüm "FARK(10, 3 - 1) = 8 (beklenen 8)" ve "2 * FARK(10, 4) = 12 (beklenen 12)" yazar. HA6'da makro doğru parantezlidir ama argüman yan etkilidir:

hata_avcisi/H09_HA6_makro_yan_etki.c · satır 15–17

```
15      int i = 3;
16      int sonuc = KARE(i++);          /* ((i++) * (i++)) */
17      printf("sonuc = %d, i = %d\n", sonuc, i);
```

Açılım `((i++) * (i++))`: `i` aynı ifadede sıralamasız iki kez değişir, **tanımsız davranış**. GCC "operation on 'i' may be undefined [-Wsequence-point]", Clang "multiple unsequenced modifications to 'i' [-Wunsequenced]" uyarısı verir (gerçek); dosya bilerek çalıştırılmadı. Düzeltme: artırmayı ayrı deyimde yapmak ya da argümanı bir kez hesaplayan fonksiyon ("kare(j++) = 9, j = 4").

HA7 · Özyinelemeli çağrının dönüş değerinin kullanılmaması

hata_avcisi/H09_HA7_donus_degeri.c · satır 10–24

```
10  int basamak_toplami(int n)
11  {
12      if (n < 10) {
13          return n;
14      }
15      n % 10 + basamak_toplami(n / 10);   /* sonuç atılıyor, return yok! */
16  }
17
18  int ebob(int a, int b)
19  {
20      if (b == 0) {
21          return a;
22      }
23      ebob(b, a % b);                 /* return unutuldu! */
24  }
```

Çağrının sonucu **yukarı taşınmıyor**: `return` unutulmuş. Değer döndürmeden biten fonksiyonun "değerini" kullanmak **tanımsız davranıştır**: GCC'li program basamak toplamı için 2 yazdı, EBOB için **şans eseri** doğru sonucu (6); Clang'lı aynı kaynak 0 ve 32703 yazdı. "Bende çalışıyor" kanıt değildir; uyarılar kanıttır. Düzeltilmiş sürüm (`return ebob(b, a % b);`) "basamak_toplami(2026) = 10 (beklenen 10)", "ebob(48, 18) = 6 (beklenen 6)" yazar.

HA7: UYARILAR VE İKİ DERLEYİCİNİN ÇALIŞTIRMASI (GERÇEK)

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H09_HA7_donus_degeri.c -o ha7
H09_HA7_donus_degeri.c:15:12: warning: value computed is not used [-Wunused-value]
H09_HA7_donus_degeri.c:16:1: warning: control reaches end of non-void function [-
Wreturn-type]
H09_HA7_donus_degeri.c:24:1: warning: control reaches end of non-void function [-
Wreturn-type]
$ ./ha7
basamak_toplami(2026) = 2  (beklenen 10)
ebob(48, 18) = 6  (beklenen 6)
$ ./ha7_clang      # aynı kaynak, clang ile
basamak_toplami(2026) = 0  (beklenen 10)
ebob(48, 18) = 32703  (beklenen 6)
```

6. Mühendis Gözüyle: Kapsamı Daralt, Yığını Hesapla

- **En dar kapsam, en kısa ömür.** Global yerine parametre ve dönüş değeri; paylaşılan durum gerekiyorsa dosya kapsamında `static` + erişim fonksiyonu (Örnek 4, Mini-OBS).
- **Arayüz bir sözleşmedir.** Başlıkta yalnızca bildirimler ve koruyucu; arayüz aynı kaldıkça gerçekleme değişse de kullanan dosyalar yalnızca yeniden bağlanır.
- **Yığın bütçesi:** derinlik × çerçeve boyu < yığın boyu. 32–64 baytlık çerçevelerle 8 MiB'ta yüz binlerce, 1 MB'ta (MSVC) on binlerce derinlik; derinliği girdiye bağlı işlerde döngü tercih edin.
- **Ölçün, sonra karar verin:** Fibonacci'de zarif kod üstel zamana mal oldu; Hanoi'de özyineleme hem zarif hem gerekliydi. **Makro yerine fonksiyon ve const.**

7. Mini-OBS v0.9: Çok Dosyalı Modüler Yapı

v0.8 fonksiyonlara bölünmüş ama 379 satırlık **tek dosyaydı**; istatistikler `main`'deki sekiz yerel değişkende tutuluyor ve `istatistik_yaz`'a **sekiz parametreyle** veriliyordu. **v0.9:** aynı davranış modüler yapıya taşınır (Şekil 9.10; kaynak `91_Proje_MiniOBS/v0.9/`, kopyası `kodlar/proje/`):

- `obs.h` — koruyucu (`OBS_H`) ve dışarıya açık 19 fonksiyonun prototipi: ekran/girdi (5), hesap kuralları (7), istatistik (7).
- `obs.c` — kurallar, girdi/rapor ve **istatistik modülü**. Kural sabitleri `static const`; istatistikler (**static kayıt sayacı** `kayit`, toplam, en yüksek/en düşük…) ve yardımcı `satir_temizle` dosya kapsamında `static`: yalnızca bu modül görür.
- `main.c` — yalnızca menü döngüsü; istatistiklere erişim fonksiyonlarıyla ulaşır. `test_obs.c` — v0.8'in 37 `assert`'i + istatistik için 7 yeni. `Makefile` — `make`, `make test`, `make clean`.

proje/obs.h · satır 28–36

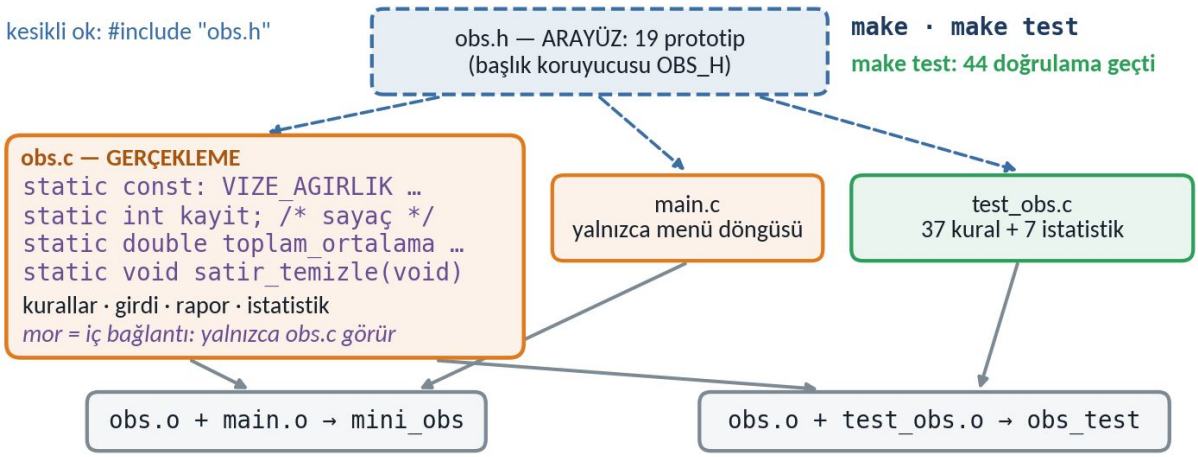
```
28  /* ---- istatistik modülü (durum obs.c'de static değişkenlerde) ---- */
29  void istatistik_sifirla(void);
30  void istatistik_ekle(long long numara, double ortalama, int yuvarlanmis,
31                       double final);
32  void istatistik_yaz(void);
33  int kayit_sayisi(void);
34  int basarili_sayisi(void);
35  int sartli_sayisi(void);
36  double sinif_ortalamasi(void);
```

proje/obs.c · satır 26–31

```
26  /* ---- İstatistik durumu: static ömür, iç bağlantı ---- */
27  static int kayit = 0;                       /* static kayıt sayacı */
28  static double toplam_ortalama = 0.0;
29  static int basarili = 0, sartli = 0;
30  static double en_yuksek = 0.0, en_dusuk = 0.0;
31  static long long en_yuksek_no = 0, en_dusuk_no = 0;
```

v0.8'de `main`'deki on beş satırlık istatistik güncellemesi aynen `istatistik_ekle`'ye taşındı; `main.c`'nin not girişi dalı artık yalnızca fonksiyon çağrılarından oluşur (`istatistik_ekle(numara, ortalama, yuvarlanmis, final);`).

Şekil 9.10
Mini-OBS v0.9 modül yapısı: obs.h arayüzü, obs.c'nin static iç durumu; main.c ve test_obs.c iki ayrı program



Derleme. Elle: `gcc -std=c17 -Wall -Wextra -pedantic -g -c obs.c`, aynısı `main.c` için, sonra `gcc obs.o main.o -o mini_obs`. `make` bağımlılıkları (`obs.o: obs.c obs.h`) izleyip yalnızca değişeni yeniden derler (gerçek oturum, `mini_obs_v0_9_ciktilar.txt`):

MAKE (GERÇEK)

```
$ make      # hiçbir şey değişmedi
make: 'mini_obs' is up to date.
$ touch obs.c && make
gcc -std=c17 -Wall -Wextra -pedantic -g -c obs.c
gcc -std=c17 -Wall -Wextra -pedantic -g obs.o main.o -o mini_obs
$ make test
gcc -std=c17 -Wall -Wextra -pedantic -g -c test_obs.c
gcc -std=c17 -Wall -Wextra -pedantic -g obs.o test_obs.o -o obs_test
./obs_test
Tüm testler geçti (44 doğrulama: 37 kural + 7 istatistik).
```

`touch obs.c` yalnızca `obs.c`'nin değişmiş gibi görünmesini sağlar; `make` yalnızca `obs.o`'yu yeniden derleyip bağladı, `main.o`'ya dokunmadı. `nm obs.o` çıktısında kural sabitleri `r`, istatistik değişkenleri `b` (küçük harf: yerel, dışarıya kapalı) olarak görünür. **Windows/MinGW:** aynı komutlar; `make` yerine `mingw32-make`, Makefile'daki `rm -f` yerine `del` gerekir.

Kabul ölçütleri. (1) Sıfır uyarı; `make` ve `make test` çalışır. (2) 44 doğrulamanın hepsi geçer. (3) Aynı girdilerle v0.8 ve v0.9 çıktıları sürüm satırı dışında **birebir aynıdır** (`diff`, Tablo 9.5). (4) `main.c` istatistik değişkenlerine doğrudan erişmez. (5) Kurallar (RTEÜ tablosu, final barajı, şartlı DC) değişmez.

Tablo 9.5
Mini-OBS v0.9 test senaryoları: v0.8 ile birebir karşılaştırma (gerçek)

Test dosyası	Senaryo	Çıkış (v0.8 / v0.9)	diff sonucu
T1 normal 4 ogrenci	4 öğrenci: BB, FF (baraj), AA, DC; özet	0 / 0	yalnızca 2. satır (sürüm)
T2 gecersiz girdiler	harf, aralık dışı menü/not, negatif numara	0 / 0	yalnızca 2. satır (sürüm)
T3 sinir degerler	59,5 → 60 (CC), final 49,9, 89,5 → 90 (AA), 49 (DD)	0 / 0	yalnızca 2. satır (sürüm)
T4 girdi yarida biter	girdi final notundan önce biter (EOF)	1 / 1	yalnızca 2. satır (sürüm)
T5 kayitsiz cikis	kayıt yokken istatistik, sonra çıkış	0 / 0	yalnızca 2. satır (sürüm)

8. Alıştırmalar ve Atölye

Atölye. (1) **Kâğıt üzerinde çağrı yığını** (20 dk): eşler `faktoriyel(4)`, `basamak_toplami(2026)` ve `hanoi(2, 'A', 'C', 'B', 1)` için çerçeveleri çizip siler; biri "işlemci", diğeri "yığın" rolündedir (PRIMM; Sentance vd., 2019). (2) **Yinelemeli ↔ özyinelemeli** (15 dk): H6–H8'den üç fonksiyonu öteki biçime çevirin. (3) **Modülerleştirme** (25 dk): Mini-OBS v0.8'i v0.9'a kendiniz taşıyıp `diff` ile doğrulayın. Dosyalar `kodlar/alistirmalar/` altındadır (Parsons ve boşluklu dosyalar bilerek derlenmez).

Alıştırma 9.1 ★☆☆ *Çıktı tahmini*

`yaz(3)` ne yazdırır? Önce çağrı yığınını çizin; her çerçevede hangi `printf` iniş, hangisi dönüş sırasında çalışıyor? (`H09_A01_cikti_tahmini.c`)

alistirmalar/H09_A01_cikti_tahmini.c · satır 7–16

```
 7  void yaz(int n)
 8  {
 9      if (n == 0) {
10          printf("| ");
11          return;
12      }
13      printf("%d ", n);
14      yaz(n - 1);
15      printf("%d ", n);
16  }
```

Alıştırma 9.2 ★★☆ *Çıktı tahmini*

Program ne yazdırır? Üç `sayac`'ın her biri için kapsamı ve ömrü yazın (`H09_A02_static_kapsam.c`).

alistirmalar/H09_A02_static_kapsam.c · satır 7–24

```c
int sayac = 10;

int artir(void)
{
    static int sayac = 0;
    sayac += 5;
    return sayac;
}

int main(void)
{
    int a = artir();
    int b = artir();
    {
        int sayac = 1;
        printf("%d ", sayac);
    }
    printf("%d %d %d\n", a, b, sayac);
```

Alıştırma 9.3 ★★☆ *Hata bul*

`2 10` girdisiyle 1024 yazan program `7 0` ile **çöküyor** (Segmentation fault). Hangi kural çiğneniyor? Düzeltin ve $x^0 = 1$'i test edin (`H09_A03_hata_bul_us.c`).

alistirmalar/H09_A03_hata_bul_us.c · satır 8–14

```c
long long us_al(long long taban, int us)
{
    if (us == 1) {
        return taban;
    }
    return taban * us_al(taban, us - 1);
}
```

Alıştırma 9.4 ★★☆ *Hata bul*

Basamak sayısını bulan fonksiyon 2026 için 4, −305 için 3 yazıyor; ama bir **sınır değerde** yanlış. Hangisi? Derleyici neden uyarmıyor? (`H09_A04_hata_bul_basamak.c`)

alistirmalar/H09_A04_hata_bul_basamak.c · satır 9–15

```c
int basamak_sayisi(int n)
{
    if (n == 0) {
        return 0;
    }
    return 1 + basamak_sayisi(n / 10);
}
```

Alıştırma 9.5 ★★☆ *Parsons*

Satırları, ikili gösterimdeki 1 bitlerini sayan özyinelemeli fonksiyona dönüşecek biçimde sıralayın ve girintileyin; bir satır fazladır. `bir_sayisi(13)` = 3 olmalı (13 = 1101). (`H09_A05_parsons_bir_sayisi.c`)

alistirmalar/H09_A05_parsons_bir_sayisi.c · satır 9–15

```
    return (n % 2) + bir_sayisi(n / 2);
}
        return 0;
unsigned int bir_sayisi(unsigned int n)
    }
    return 1;
    if (n == 0) {
```

Alıştırma 9.6 ★★☆ *Kod tamamla*

Boşlukları doldurun: basamaklar aralarında boşlukla, **soldan sağa** yazılsın (2026 → `2 0 2 6`, 7 → `7`). İş çağrıdan önce mi, sonra mı yapılmalı? (`H09_A06_kod_tamamla_basamaklar.c`)

alistirmalar/H09_A06_kod_tamamla_basamaklar.c · satır 8–16

```
void basamaklari_yaz(int n)
{
    if (n < ____) {
        printf("%d", n);
        return;
    }
    basamaklari_yaz(____);
    printf(" %d", ____);
}
```

Alıştırma 9.7 ★☆☆ *Kod yaz*

`artan(a, b)` ve `azalan(a, b)` fonksiyonlarını **döngüsüz** yazın: a'dan b'ye sayıları artan ve azalan sırada yazdırsınlar. İki fonksiyon arasındaki tek fark ne? Test: `3 7` → `3 4 5 6 7` ve `7 6 5 4 3`; a > b ise hiçbir şey yazılmaz.

Alıştırma 9.8 ★★☆ *Kod yaz*

Bir sayıyı özyinelemeyle ters çeviren `long long ters(long long n, long long birikim)` fonksiyonunu yazın (`ters(n, 0)` ile çağrılır) ve sayının palindrom olup olmadığını söyleyin (H6, Örnek 10'un özyinelemeli karşılığı). Test: 12321 → palindrom; 1230 → 321. Kuyruk özyinelemeli mi?

Alıştırma 9.9 ★★☆ *Kod yaz*

H6'daki Collatz adım sayısını (`H06_14_collatz.c`) özyinelemeli `int adim(long long n)` olarak yazın: `adim(1) = 0`; n çiftse `1 + adim(n / 2)`, tekse `1 + adim(3 * n + 1)`. 27 için H6 ile aynı sonucu (111) bulun. Bu fonksiyonun her girdide temel duruma ulaşacağını kanıtlayabilir misiniz?

Alıştırma 9.10 ★★☆ *Kod yaz*

Çok dosyalı modül. `sicaklik.h` / `sicaklik.c` / `main.c` ve bir `Makefile` yazın: `c_to_f`, `f_to_c` ve `donusum_sayisi()`; sayaç `sicaklik.c`'de `static` olsun. `nm sicaklik.o` çıktısında sayacın harfi ne olmalı? Test: 100 °C = 212 °F, −40 °C = −40 °F.

Alıştırma 9.11 ★★★ *Kod yaz*

Merdiven. n basamaklı bir merdiveni her adımda 1 ya da 2 basamak atlayarak kaç farklı yolla çıkabilirsiniz? `yol(n) = yol(n − 1) + yol(n − 2)`; temel durumları siz belirleyin. Özyinelemeli sürümün çağrı sayısını `static` bir sayaçla ölçün (n = 5, 10, …, 30) ve yinelemeli sürümle karşılaştırın. Hangi örneğe benziyor?

Alıştırma 9.12 ★★★ *Kod yaz*

H7'deki yıldız üçgenini **hiç döngü kullanmadan** çizin: `satir(k)` k yıldız yazsın; `ucgen(n)` 1'den n'ye, `ters_ucgen(n)` n'den 1'e satırlar çizsin. İkisinin farkı, Örnek 8'deki hangi fikirdir?

9. Kendini Değerlendirme

Yapabiliyorum…	
☐ Bir adın kapsamını ve neyi gölgelediğini söyleyebiliyorum.	☐ Otomatik ve statik ömrü ayırt edip static sayaç yazabiliyorum.
☐ .h / .c modülü yazıp gcc -c ve bağlamayla derleyebiliyorum.	☐ Derleyici hatasını bağlayıcı hatasından ayırt edebiliyorum.
☐ Makro açılımını gcc -E ile görüp tuzaklarını açıklayabiliyorum.	☐ Süreç bellek düzenini çizip değişkenleri yerleştirebiliyorum.
☐ Özyinelemeli çağrıların yığın çerçevelerini push/pop sırasıyla çizebiliyorum.	☐ Temel durumu, adımı ve temel duruma yaklaşmayı gösterebiliyorum.
☐ Aynı problemi yinelemeli ve özyinelemeli çözüp karşılaştırabiliyorum.	☐ fib(n)'in neden yavaş, Hanoi'nin neden $2^n - 1$ hamle olduğunu açıklayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **Fibonacci'nin bedeli ve böl ve fethet**

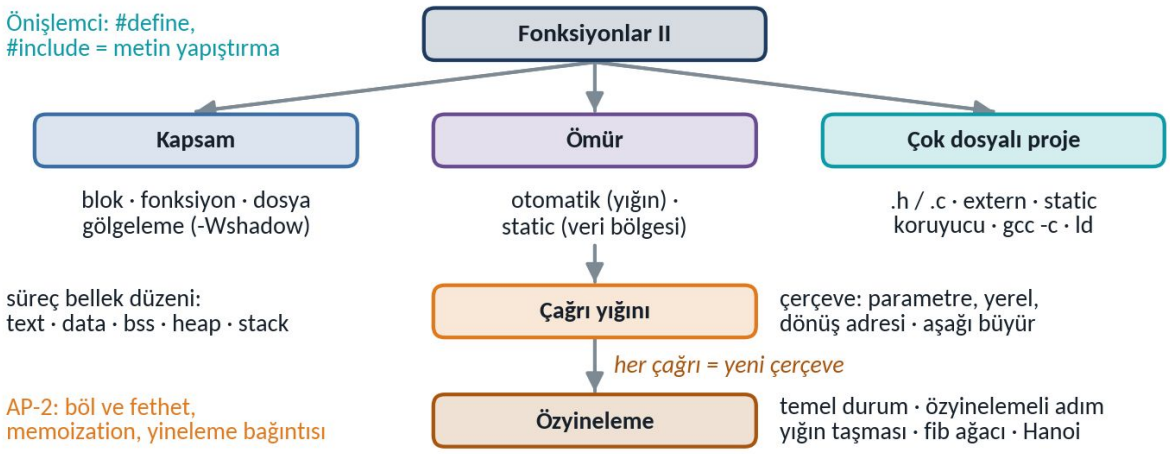
- **Memoization ve dinamik programlama.** Örnek 12'deki tekrarın ilacı, her fib(k)'yı **ilk hesaplandığında not edip** sonra notu okumaktır (**memoization**); alt problem çözümlerini bir tabloda biriktirmek **dinamik programlamadır** (AP-2). Sezgisi: ağaçtaki kırmızı düğümleri hiç hesaplamamak.
- **Çağrı sayısını hesaplamak.** Her n için çağrı sayısı = 2 · fib(n + 1) − 1 (n = 40: 2 · 165 580 141 − 1 = 331 160 281; bütün ölçümlerde sınadık). Kaynağı C(n) = C(n − 1) + C(n − 2) + 1, C(0) = C(1) = 1 sayımıdır; böyle eşitliklere **yineleme bağıntısı** (recurrence relation) denir, Hanoi'nin H(n) = 2H(n − 1) + 1 kuralı da böyledir (AP-2).
- **Böl ve fethet.** Hanoi problemi aynı türden küçük problemlere bölüp birleştirir; AP-2'deki **birleştirme sıralaması**, **hızlı sıralama** ve **özyinelemeli ikili arama** bu düşünceyle çalışır.

> MERAK KUTUSU · **Yığın ne kadar derin olabilir? Kendi makinenizde ölçün**
>
> Örnek 11'i `-DARALIK=1000` ile derleyip farklı `ulimit -s` değerleriyle çalıştırın (Linux/WSL). `dal`'a birkaç yerel değişken ekleyin: çöküş derinliği ve çerçeve boyu nasıl değişti? `-O2` ile ne olur, neden (Bölüm 3.8)? Windows'ta yığın boyu bağlama sırasında yazılır (MSVC'de `/STACK`; Microsoft, t.y.).

11. Özet, Kavram Haritası ve Terim Sözlüğü

Özet. Kapsam adın **nerede**, ömür değişkenin **ne zaman** var olduğunu söyler; `static` yerelde ömrü uzatır, dosya kapsamında adı modüle hapseder. `.h` arayüz, `.c` gerçeklemedir; `#include` metin yapıştırır, `gcc -c` her dosyayı ayrı derler, bağlayıcı `U` sembollerini tanımlarla eşler. Makro metin değiştirmedir. Her çağrı yığında parametre, yerel ve dönüş adresi içeren bir çerçeve açar; yığın sınırlıdır. Özyineleme = temel durum + küçülen özyinelemeli adım; çağrıdan sonraki iş ters sırada yapılır.

Şekil 9.11
Haftanın kavram haritası



Büyük Sorunun yanıtı. Fonksiyon kendini çağırınca sıradan bir çağrıda olan olur: yığında **yeni bir çerçeve** açılır ve parametreleri, yerelleri öncekilerden **ayrıdır**; faktoriyel(4) çalışırken dört ayrı `n` aynı anda yığında durur (64 bayt arayla). Her `call` dönüş adresini yığına iter, her `ret` en üsttekini çeker; LIFO düzeni dönüşlerin çağrıların tersi sırasıyla, karışmadan gerçekleşmesini sağlar. Bedeli de ölçüldü: temel durumu olmayan özyineleme yaklaşık 261 000 çağrıda 8 MiB'lık yığını tüketti; fib(40) 331 160 281 çağrı yaptı; Hanoi ise 64 diskte evrenin yaşının 42,4 katı süren bir işi tek fonksiyonla tarif etti.

Terim sözlüğü

Tablo 9.6
Haftanın terimleri (yol haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Kısa açıklama
kapsam; gölgeleme	scope; shadowing	Adın görünür olduğu bölge; iç bildirimin dıştakini gizlemesi
ömür (saklama süresi)	lifetime (storage duration)	Değişkenin bellekte var olduğu süre: otomatik ya da statik
global değişken; yan etki	global variable; side effect	Dosya kapsamındaki değişken; fonksiyonun dışarıda bıraktığı değişiklik
başlık dosyası; başlık koruyucusu	header file; include guard	Arayüz (.h); #ifndef/#define/#endif
çeviri birimi; nesne dosyası	translation unit; object file	Başlıklarıyla birlikte bir .c; derlenmiş .o

Türkçe	İngilizce	Kısa açıklama
iç / dış bağlantı	internal / external linkage	static ile yalnızca dosyada; varsayılan olarak tüm programda görünürlük
bildirim; tanım	declaration; definition	"Bu ad var" ile "bu ad için bellek/gövde burada"
önişlemci; makro	preprocessor; macro	Derlemeden önce metin işleyen aşama; #define ile metin değiştirme
çağrı yığını; yığın çerçevesi; dönüş adresi	call stack; stack frame; return address	Etkin çağrıların LIFO yığını; bir çağrının parametre, yerel ve dönüş adresi (çağrıdan sonraki komut) bölümü
yığın / öbek	stack / heap	Otomatik değişkenler / malloc ile alınan bellek (H12)
özyineleme; temel durum; yığın taşması	recursion; base case; stack overflow	Fonksiyonun kendini çağırması; çağrı yapmadan dönülen durum; çerçevelerin yığın sınırını aşması
kuyruk özyinelemesi	tail recursion	Özyinelemeli çağrının son iş olduğu biçim

Gelecek hafta: Diziler I. Aynı türden çok sayıda veriyi tek adla, **bitişik bellekte** saklayacağız (`int notlar[100];`). H7'deki onluktan ikiliye dönüşümü bu hafta yığınla çözdük; H10'da bitleri bir **dizide** biriktireceğiz. Mini-OBS v0.10 öğrencileri ilk kez **saklayacak** (paralel diziler). Hazırlık: bir dizinin hangi bellek bölgesinde yer alacağını tahmin edin.

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Free Software Foundation. (2020). *GCC 10 release series: Changes, new features, and fixes*. https://gcc.gnu.org/gcc-10/changes.html

Free Software Foundation. (t.y.-a). Options to request or suppress warnings. In *Using the GNU Compiler Collection (GCC)*. https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

Free Software Foundation. (t.y.-b). Command-line options. In *The GNU linker (ld)*. https://sourceware.org/binutils/docs/ld/Options.html

Hinz, A. M., Klavžar, S., & Petr, C. (2018). *The Tower of Hanoi – Myths and maths* (2nd ed.). Birkhäuser. https://doi.org/10.1007/978-3-319-73779-9

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Merrotsy, P. (2015). The Tower of Hanoi and inductive logic. *Australian Senior Mathematics Journal*, *29*(1), 16–24.

Microsoft. (t.y.). */STACK (Stack allocations)*. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/build/reference/stack-stack-allocations

Planck Collaboration. (2020). Planck 2018 results. VI. Cosmological parameters. *Astronomy & Astrophysics*, *641*, Article A6. https://doi.org/10.1051/0004-6361/201833910

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education*, *29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781